Conceptos de Bases de Datos Relacionales

El **modelo relacional** es el más popular para bases de datos. Su éxito se debe a que permite que los datos sean **independientes**, es decir, que su organización no depende de cómo se usen o almacenen. Los datos se guardan en estructuras simples llamadas **tablas**. Esto nos da flexibilidad, ya que no importa si cambiamos la forma en que los datos se guardan en el disco o cómo se organizan lógicamente.

Un **modelo entidad-relación (ER)** es otra forma de ver los datos. Aunque no es un modelo de base de datos por sí mismo, es una herramienta clave para **diseñar bases de datos relacionales**.

Imagina una biblioteca. Un diagrama ER nos ayudaría a visualizar cómo se relacionan los diferentes elementos:

- **Libros:** Tienen un título, una edición, un año de escritura, etc.
- **Autores:** Pueden escribir uno o varios libros.
- **Copias:** La biblioteca puede tener varias copias del mismo libro.
- **Prestatarios:** Cada copia solo puede ser prestada por una persona a la vez.

Entidades y Atributos

En un modelo ER, pensamos en la base de datos como una colección de **entidades**. Una **entidad** es cualquier "cosa" o "sustantivo" que existe por sí misma en nuestra base de datos, como un libro, una persona o un lugar. En un diagrama ER, las entidades se representan con un **rectángulo**.

Las entidades tienen **atributos**, que son las características o propiedades que las describen. Por ejemplo, los atributos de un libro serían su **título**, su **edición** o el **año** en que se escribió. En un diagrama ER, los atributos se dibujan como **óvalos** conectados a su entidad.

De Entidades a Tablas

La magia ocurre cuando transformamos este diseño ER en una base de datos relacional:

- Cada **entidad** se convierte en una **tabla** en la base de datos. Así, la entidad "libro" se transforma en la tabla "Libros".
- Los **atributos** de la entidad se convierten en las **columnas** de esa tabla. Por ejemplo, la tabla "Libros" tendrá columnas como "Título", "Edición" y "Año".

Las tablas son una combinación de filas y columnas. Cada columna almacena un tipo específico de dato:

- **VARCHAR:** Para textos que varían en longitud, como un título de libro.
- **CHAR:** Para textos de longitud fija.
- **Numérico:** Para números, como el año de publicación.

- **CHAR (con guiones):** Para códigos como el ISBN, que combinan números y guiones.

Usando este proceso, podemos crear tablas para todos los elementos de nuestra biblioteca: "Autores", "Lista de Autores", "Prestatarios", "Préstamo" y "Copia", con sus respectivos atributos como columnas.

Claves: La Identificación y Conexión en las Tablas

Para que las tablas funcionen correctamente, necesitamos dos tipos de claves:

- **Clave Principal (Primary Key):** Es un atributo o conjunto de atributos que **identifica de forma única** cada fila (o registro) en una tabla. Esto significa que no puede haber dos filas con la misma clave principal. Evita que tengamos datos duplicados y es fundamental para establecer relaciones. Cada tabla tiene una clave principal.
- **Clave Externa (Foreign Key):** Es una clave principal de otra tabla que usamos en la tabla actual para **vincularlas**. Por ejemplo, en una tabla de "Préstamos", **podríamos tener una clave externa que apunte al "ID de Libro" en la tabla "Libros" y al "ID de Prestatario**" en la tabla "Prestatarios", conectando así quién prestó qué libro.

--- ---

Tipos de Sentencias SQL: DDL vs. DML

Las sentencias **SQL** son comandos que usamos para interactuar con los componentes de una base de datos relacional: las **tablas**, sus **columnas** (que son los atributos) y las **filas** (que contienen los datos).

Podemos dividir las sentencias SQL en dos categorías principales:

Lenguaje de Definición de Datos (DDL)

Las sentencias **DDL** (Data Definition Language) se utilizan para **definir, modificar o eliminar la estructura de la base de datos y sus objetos**, como las tablas. Piensa en DDL como las herramientas para "construir" o "remodelar" la base de datos.

Los tipos comunes de sentencias DDL incluyen:

- **`CREATE`**: Se usa para **crear nuevas tablas** y definir sus columnas (nombre, tipo de dato, etc.). Por ejemplo, `CREATE TABLE Libros (...).`
- **`ALTER`**: Permite **modificar la estructura de una tabla existente**. Esto incluye añadir o eliminar columnas, o cambiar el tipo de dato de una columna. Por ejemplo, `ALTER TABLE Libros ADD COLUMN ISBN VARCHAR(13).`

- **`TRUNCATE`**: Se usa para **eliminar rápidamente todos los datos de una tabla**, pero mantiene la tabla intacta (no la borra del todo). Es como vaciar un contenedor sin tirarlo.
- **`DROP`**: Se utiliza para **eliminar completamente una tabla** (o cualquier otro objeto de la base de datos). Esto borra tanto los datos como la estructura de la tabla. Por ejemplo, `DROP TABLE Libros`.

Lenguaje de Manipulación de Datos (DML)

Las sentencias **DML** (Data Manipulation Language) se utilizan para **leer y modificar los datos que están dentro de las tablas**. A menudo se les conoce como operaciones **CRUD**, un acrónimo que significa: **C**rear, **R**eer, **U**pdate (actualizar) y **D**elete (eliminar filas) en una tabla. Piensa en DML como las herramientas para "gestionar el contenido" de tu base de datos.

Los tipos comunes de sentencias DML incluyen:

- **`INSERT`**: Se usa para **añadir una o varias filas de datos** en una tabla. Por ejemplo, `INSERT INTO Libros (Titulo) VALUES ('Cien años de soledad')`.
- **`SELECT`**: Permite **leer o seleccionar filas de datos** de una o varias tablas. Es la sentencia más utilizada y te permite buscar y recuperar información. Por ejemplo, `SELECT * FROM Libros`.
- **`UPDATE`**: Se utiliza para **modificar los datos de una o varias filas** en una tabla. Por ejemplo, `UPDATE Libros SET Edicion = 'Primera' WHERE Titulo = 'Cien años de soledad'`.
- **`DELETE`**: Se usa para **eliminar una o varias filas de datos** de una tabla. Por ejemplo, `DELETE FROM Libros WHERE Titulo = 'La sombra del viento'`.

--- ---

Sentencia CREATE TABLE

La sentencia `CREATE TABLE` es el comando **DDL** más común que usamos para **construir la estructura de nuestras tablas** en una base de datos. Nos permite definir el nombre de la tabla y las características de cada una de sus columnas.

sintaxis básica

```
CREATE TABLE NombreDeLaTabla (
      nombre_columna1 TIPO_DE_DATO [OPCIONES],
      nombre_columna2 TIPO_DE_DATO [OPCIONES],
      -- ... más columnas
PRIMARY KEY (nombre_columna_clave_primaria) -- Opcional, si la clave
primaria es una sola columna);
```

Dentro de los paréntesis, definimos cada **columna**. Por cada columna, debemos especificar:

1. El **nombre** de la columna.
2. El **tipo de dato** que almacenará (por ejemplo, texto, números, fechas).
3. **Opciones adicionales** (como si es una clave primaria o si no puede estar vacía).

Cada definición de columna debe ir separada por una **coma**.

Ejemplo Simple: Tabla de Provincias

Imaginemos que queremos crear una tabla para almacenar información sobre las provincias de Canadá. Podríamos hacerlo así:

```
CREATE TABLE Provincias (
ID CHAR(2) PRIMARY KEY NOT NULL,
Nombre VARCHAR(24)
);
```

```
--CREATE TABLE Provincias: Indica que vamos a crear una tabla llamada
Provincias.
--ID CHAR(2)PRIMARY KEY NOT NULL:
```

- ID: Es el nombre de la columna.
- CHAR(2): Es el tipo de dato. CHAR significa "cadena de caracteres de longitud fija". En este caso, de 2 caracteres (perfecto para códigos como 'AB', 'BC').
- PRIMARY KEY: Define esta columna como la clave primaria de la tabla, lo que garantiza que cada ID sea único y no se repita.
- NOT NULL: Significa que esta columna no puede quedar vacía; siempre debe tener un valor.

```
--Nombre VARCHAR(24):
```

- Nombre: Es el nombre de la segunda columna.
- VARCHAR(24): Es el tipo de dato VARCHAR, que significa "cadena de caracteres de longitud variable". Puede almacenar hasta 24 caracteres, ajustándose al texto que se necesite (como "Alberta", "Columbia Británica").

Al ejecutar esta sentencia, se crea una tabla Provincias con dos columnas: ID (para códigos de provincia) y Nombre (para el nombre completo de la provincia).

Ejemplo Completo

Ahora, un ejemplo más detallado basado en una base de datos de biblioteca. Queremos crear una tabla para la entidad Autor, que tendrá columnas como author_id, apellido, nombre, correo_electronico y ciudad.

```
CREATE TABLE Autor (
    author_id CHAR(2) PRIMARY KEY NOT NULL,
    apellido VARCHAR(15) NOT NULL,
    nombre VARCHAR(15) NOT NULL,
    correo_electronico VARCHAR(40),
    ciudad VARCHAR(15),
    pais CHAR(2)
);
```

--- ---

ALTERAR, ELIMINAR Y TRUNCAR TABLAS

La sentencia ALTER TABLE se utiliza para agregar o eliminar columnas de una tabla, modificar el tipo de datos de las columnas, agregar o eliminar claves (primarias o foráneas), y añadir o quitar restricciones.

Sintaxis básica

La instrucción comienza con ALTER TABLE, seguida del nombre de la tabla que se desea modificar. A diferencia de CREATE TABLE, no se utilizan paréntesis para encerrar los parámetros. Cada línea dentro de la sentencia ALTER TABLE representa un cambio que se desea aplicar.

Ejemplo 1: Agregar una columna

Para agregar una columna llamada `telefono_numero` a la tabla `autor` (en una base de datos de biblioteca), puedes usar la siguiente sentencia:

```
ALTER TABLE autor ADD COLUMN telefono_numero BIGINT;
```

-- `Modificar el tipo de datos de una columna`

Si necesitas permitir caracteres como paréntesis, signos de más o guiones en los números telefónicos, puedes cambiar el tipo de dato de telefono_numero a CHAR(20):

```
ALTER TABLE autor MODIFY telefono_numero CHAR(20);
```

⚠ ***Advertencia:*** *Modificar el tipo de datos de una columna que ya contiene datos puede causar errores si los valores existentes no son compatibles con el nuevo tipo. Por ejemplo, no puedes convertir una columna con texto a tipo numérico si contiene datos no numéricos.*

-- `Eliminar una columna`

Si ya no necesitas la columna telefono_numero, puedes eliminarla con la siguiente sentencia:

```
ALTER TABLE autor DROP COLUMN telefono_numero;
```

```
-- Eliminar Tabla completa
```

Si deseas eliminar toda una tabla, utiliza la sentencia DROP TABLE. Esto eliminará tanto la estructura como los datos de la tabla:

```
DROP TABLE autor;
```

```
-- Eliminar todos los datos de una tabla sin eliminar la tabla
```

Si solo quieres borrar el contenido de una tabla sin eliminar su estructura, puedes usar:

- DELETE FROM nombre_tabla; → pero esto puede ser más lento.
- TRUNCATE TABLE nombre_tabla IMMEDIATE; → elimina todos los registros de forma más rápida y no se puede deshacer.

```
TRUNCATE TABLE autor IMMEDIATE;
```

--- ---

Scripts SQL

Los scripts SQL son una serie de comandos o un programa que se ejecutará en un servidor SQL.

Los scripts SQL son útiles para realizar cambios complejos en la base de datos y se pueden utilizar para crear, modificar o eliminar objetos de la base de datos, como tablas, vistas, procedimientos almacenados y funciones.

Aplicaciones de Scripts SQL

Aquí hay algunas de las cosas que puedes hacer con scripts SQL:

- **Crear tablas**
  Puedes usar scripts SQL para crear nuevas tablas en tu base de datos. Esto es útil cuando necesitas agregar nueva funcionalidad a tu aplicación o cuando deseas almacenar nuevos tipos de datos.
- **Eliminar tablas**
  Los scripts SQL a menudo tienen comandos para eliminar tablas de las bases de

datos. Esto es especialmente importante antes de los comandos de creación de tablas para asegurarte de que no exista ya una tabla con el mismo nombre en la base de datos.

- **Insertar datos**
  Los scripts SQL también se pueden utilizar para insertar datos en tus tablas. Esto es útil cuando necesitas poblar tu base de datos con datos de prueba o cuando deseas importar datos de una fuente externa.
- **Actualizar datos**
  Puedes usar scripts SQL para actualizar datos existentes en tus tablas. Esto es útil cuando necesitas corregir errores o actualizar registros según los cambios en los requisitos del negocio.
- **Eliminar datos**
  Los scripts SQL también se pueden utilizar para eliminar datos de tus tablas. Esto es útil cuando necesitas eliminar registros antiguos u obsoletos de tu base de datos.
- **Crear vistas**
  Las vistas son tablas virtuales que te permiten consultar datos de múltiples tablas como si fueran una sola tabla. Puedes usar scripts SQL para crear vistas que simplifiquen consultas complejas y faciliten el trabajo con tus datos.
- **Crear procedimientos almacenados**
  Los procedimientos almacenados son declaraciones SQL precompiladas que se pueden ejecutar bajo demanda. Puedes usar scripts SQL para crear procedimientos almacenados que encapsulen lógica de negocio compleja y faciliten la gestión de tu base de datos.
- **Crear disparadores**
  Los disparadores son tipos especiales de procedimientos almacenados que se ejecutan automáticamente en respuesta a ciertos eventos, como una operación de inserción, actualización o eliminación. Puedes usar scripts SQL para crear disparadores que apliquen reglas de negocio y mantengan la integridad de los datos.

Ejemplo: Creación de Tablas

Ejecutemos un script que contenga los comandos CREATE TABLE para todas las tablas en un conjunto de datos dado, en lugar de crear cada tabla manualmente escribiendo los comandos DDL en el editor SQL.

Tenga en cuenta los siguientes puntos sobre estos scripts.

1. Los scripts SQL son básicamente un conjunto de comandos SQL compilados en un solo archivo.
2. Cada comando debe terminarse con un delimitador o terminador. A menudo, el delimitador predeterminado es un punto y coma;.
3. Se recomienda mantener la extensión del archivo como `.sql`.
4. Al importar este archivo en la interfaz de phpMyAdmin, los comandos en el archivo se ejecutan secuencialmente.

Considere el siguiente guion;

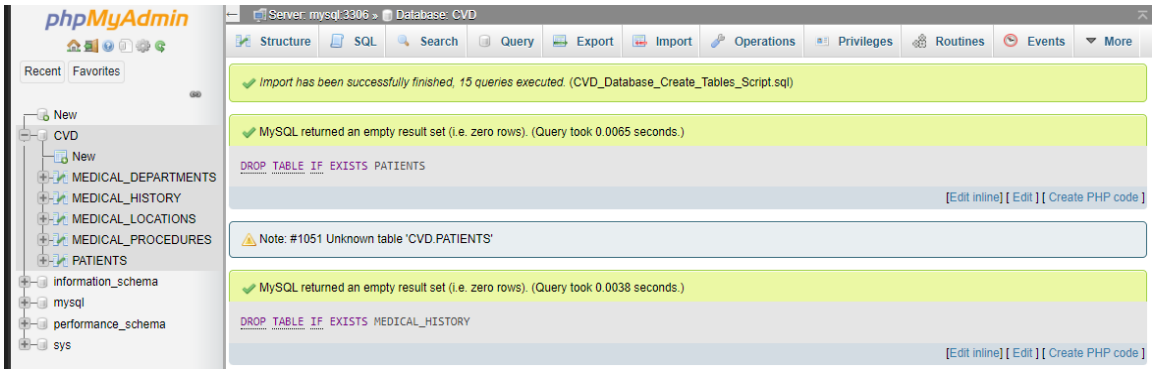
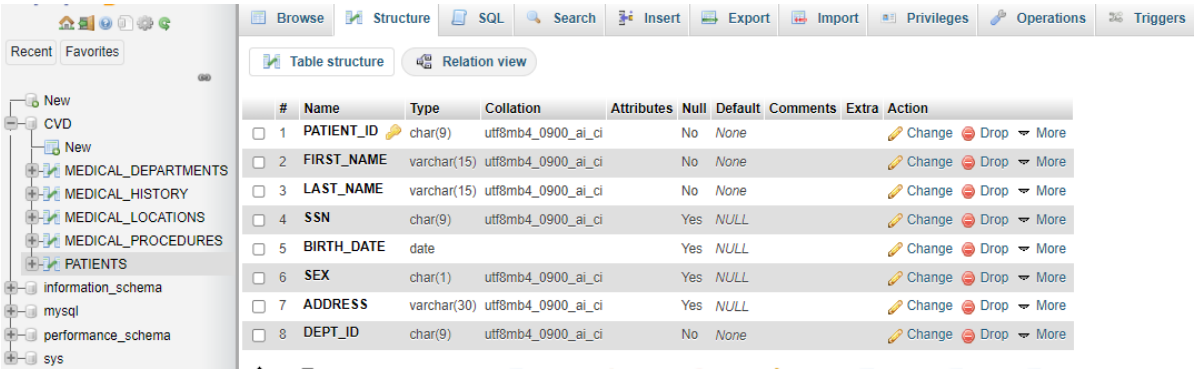
```
DROP TABLE IF EXISTS PATIENTS;
DROP TABLE IF EXISTS MEDICAL_HISTORY;
DROP TABLE IF EXISTS MEDICAL_PROCEDURES;
DROP TABLE IF EXISTS MEDICAL_DEPARTMENTS;
DROP TABLE IF EXISTS MEDICAL_LOCATIONS;


CREATE TABLE PATIENTS (
  PATIENT_ID CHAR(9) NOT NULL,
  FIRST_NAME VARCHAR(15) NOT NULL,
  LAST_NAME VARCHAR(15) NOT NULL,
  SSN CHAR(9),
  BIRTH_DATE DATE,
  SEX CHAR,
  ADDRESS VARCHAR(30),
  DEPT_ID CHAR(9) NOT NULL,
  PRIMARY KEY (PATIENT_ID)
);

CREATE TABLE MEDICAL_HISTORY (
  MEDICAL_HISTORY_ID CHAR(9) NOT NULL,
  PATIENT_ID CHAR(9) NOT NULL,
  DIAGNOSIS_DATE DATE,
  DIAGNOSIS_CODE VARCHAR(10),
  MEDICAL_CONDITION VARCHAR(100),
  DEPT_ID CHAR(9),
  PRIMARY KEY (MEDICAL_HISTORY_ID)
);

CREATE TABLE MEDICAL_PROCEDURES (
  PROCEDURE_ID CHAR(9) NOT NULL,
  PROCEDURE_NAME VARCHAR(30),
  PROCEDURE_DATE DATE,
  PATIENT_ID CHAR(9) NOT NULL,
```

```
  DEPT_ID CHAR(9),
  PRIMARY KEY (PROCEDURE_ID)
);

CREATE TABLE MEDICAL_DEPARTMENTS (
  DEPT_ID CHAR(9) NOT NULL,
  DEPT_NAME VARCHAR(15),
  MANAGER_ID CHAR(9),
  LOCATION_ID CHAR(9),
  PRIMARY KEY (DEPT_ID)
);

CREATE TABLE MEDICAL_LOCATIONS (
  LOCATION_ID CHAR(9) NOT NULL,
  DEPT_ID CHAR(9) NOT NULL,
  LOCATION_NAME VARCHAR(50),
  PRIMARY KEY (LOCATION_ID, DEPT_ID)
);
```

Este script incorpora comandos para eliminar primero las tablas con los nombres mencionados en la base de datos. Posteriormente, el script contiene comandos para crear cinco tablas diferentes. Todos estos comandos se ejecutan secuencialmente en la interfaz.

El contenido de este archivo se puede guardar en un .sql formato de archivo y ejecutarse en la interfaz de phpMyAdmin. Para ello, primero se selecciona la base de datos, se carga el script SQL en el espacio proporcionado y se ejecuta, como se muestra en la imagen a continuación.

Tras la ejecución correcta de cada instrucción en secuencia, aparece una nota en la interfaz, como se muestra en la imagen a continuación. Cabe destacar que las tablas creadas ahora son visibles en la estructura de árbol a la izquierda, debajo de la base de datos seleccionada.



Puede hacer clic en cualquiera de las tablas para ver su definición (lista de columnas, tipos de datos, etc.). La imagen a continuación muestra la estructura de la tabla PACIENTES.